

COMP 271 · DATA STRUCTURES

Recursion

Solving a problem by solving a smaller version of itself

DR. KARIMA ENNAOUI

LOYOLA UNIVERSITY CHICAGO

Plan

- What Is Recursion? – base cases, recursive cases, and the call stack.
- How Do We Measure “Fast”? – a short, intuitive detour on cost.
- Divide and Conquer: Binary Search – recursion that shrinks fast.
- When Recursion Misbehaves: Naive Fibonacci – recursion that doesn’t.
- Writing Your Own Recursive Functions – a practical checklist.



PART 1

What Is Recursion?

Base cases, recursive cases, and the call stack

Loops vs. Recursion

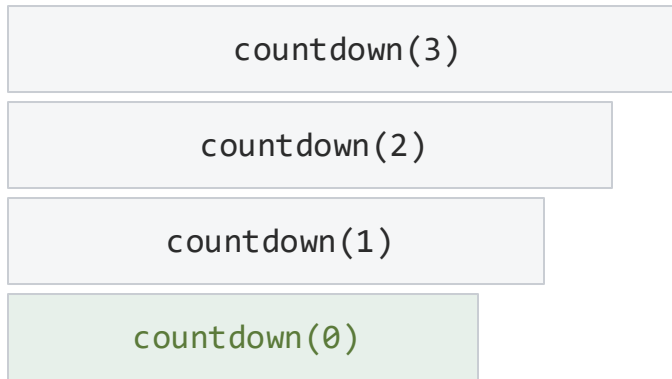
- A loop repeats by updating a variable until a condition fails.
- Recursion repeats by calling the same function again, on a smaller version of the same problem.
- **Today: how to design a recursive function, trace what it does, and reason about its cost.**

countdown(n)

- No return value: this function only prints.
- **Base case stops the recursion.**
- **Recursive case calls countdown again on $n - 1$, a smaller input.**

```
def countdown(n):  
    if n <= 0:  
        print("Liftoff!")  
        return  
  
    print(n)  
    countdown(n - 1)
```

Tracing countdown(3)



Output, in order: 3, 2, 1, Liftoff!

- Each call is added to the stack: this is called winding.
- `countdown(0)` hits the base case and prints “Liftoff!”
- Calls then pop off the stack in reverse order: unwinding.

This is last-in, first-out (LIFO) behavior: the same discipline you’ll meet formally, later this semester, as the Stack ADT.

Every Recursive Function Needs Three Things

- **Base case** – a situation simple enough to answer directly, with no further recursive call.
- **Recursive case** – reduce the problem and call the function again.
- **Progress** – every recursive call must move closer to a base case.

Miss any one of these and recursion never stops: Python raises a `RecursionError` once the call stack is exhausted.

factorial(n)

```
def factorial(n):  
    if n == 0:  
        return 1  
  
    return n * factorial(n - 1)
```

- Unlike countdown, this function returns a value.
- The multiplication happens on the way back, after the recursive call returns.

Tracing factorial(4)

Call	Waiting to compute	Returns
factorial(4)	$4 \times \text{factorial}(3)$	24
factorial(3)	$3 \times \text{factorial}(2)$	6
factorial(2)	$2 \times \text{factorial}(1)$	2
factorial(1)	$1 \times \text{factorial}(0)$	1
factorial(0)	base case	1

- Calls wind down to factorial(0), the base case, first.
- Multiplications then happen while unwinding back up the stack.

Checkpoint 1: Trace It Yourself

```
def sum_to(n):  
    if n == 0:  
        return 0  
    return n + sum_to(n - 1)
```

- What is the base case? The recursive case?
- What does the call stack look like at its deepest point?
- What value does `sum_to(3)` return?

Two minutes, on your own or with a neighbor. Be ready to walk the class through your trace.

PART 2

How Do We Measure “Fast”?

A short, intuitive detour before we look at cost

Why We Care How “Fast” Code Is

- Two pieces of code can both be correct and still behave very differently as input grows.

Why We Care How “Fast” Code Is

- Two pieces of code can both be correct and still behave very differently as input grows.
- "contains" checks each element once, in the worst case: n comparisons for a list of length n .
- **We write this as $O(n)$: “on the order of n steps.”**

```
def contains(values, target):  
    found = False  
    for value in values:  
        if value == target:  
            found = True  
    return found
```

Why We Care How “Fast” Code Is

- Two pieces of code can both be correct and still behave very differently as input grows.
- "contains" checks each element once, in the worst case: n comparisons for a list of length n .
- **We write this as $O(n)$: “on the order of n steps.”**
- $O()$ notation ignores constants: $3n + 7$ and n are both just $O(n)$.

```
def contains(values, target):  
    found = False  
    for value in values:  
        if value == target:  
            found = True  
    return found
```

Not All Growth Is Created Equal

n	$O(1)$	$O(\log n)$	$O(n)$	$O(n^2)$
10	1	≈ 3	10	100
100	1	≈ 7	100	10,000
1000	1	≈ 10	1000	1,000,000

Watch the $O(\log n)$ column: that's where binary search lands, in the next section.

PART 3

Divide and Conquer: Binary Search

A second classic example: recursion that shrinks fast

Guess My Number, 1–100?

Guess My Number, 1–100

- Best strategy: always guess the middle of what's left.
- “Too high” or “too low” eliminates about half the remaining numbers.
- **You can find any number in 100 in at most 7 guesses.**

Guess My Number, 1–100

- Best strategy: always guess the middle of what's left.
 - “Too high” or “too low” eliminates about half the remaining numbers.
 - **You can find any number in 100 in at most 7 guesses.**
- => Binary search applies this exact idea to searching a sorted array.

Search a Sorted Array

- Given a sorted array and a target value, return the target's index (or -1 if absent).

3	8	12	17	23	31	42	56	70
0	1	2	3	4	5	6	7	8

target = 42

Search a Sorted Array

- Given a sorted array and a target value, return the target's index (or -1 if absent).

3	8	12	17	23	31	42	56	70
0	1	2	3	4	5	6	7	8

target = 42

A linear scan checks values one at a time: $O(n)$, as we just saw. Can sorted order help us skip values instead?

Binary Search: Examine, Compare, Shrink

- Examine the middle value.
- If Match → stop; Else
- If Target smaller → search the left half; Else
- If Target larger → search the right half.

=> Every step discards about half of the remaining search space.

Binary Search: Examine, Compare, Shrink

- Examine the middle value.
 - If Match → stop; Else
 - If Target smaller → search the left half; Else
 - If Target larger → search the right half.
- => Every step discards about half of the remaining search space.

3	8	12	17	23	31	42	56	70
0	1	2	3	4	5	6	7	8

target = 42

Call	Active interval	Middle	Comparison	Next action
1	0..8	4 → 23	42 > 23	Search 5..8
2	5..8	6 → 42	Equal	Return 6

Binary Search: Examine, Compare, Shrink

- Examine the middle value.
 - If Match → stop; Else
 - If Target smaller → search the left half; Else
 - If Target larger → search the right half.
- => Every step discards about half of the remaining search space.

3	8	12	17	23	31	42	56	70
0	1	2	3	4	5	6	7	8

target = 42

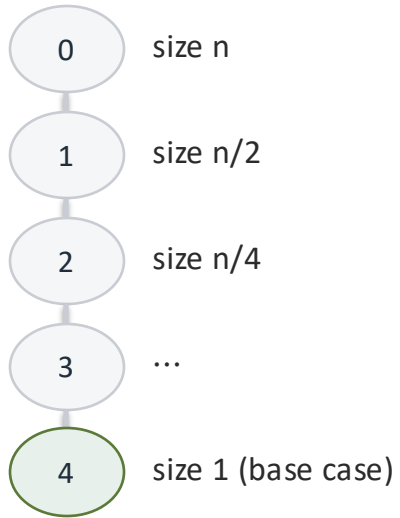
Call	Active interval	Middle	Comparison	Next action
1	0..8	4 → 23	42 > 23	Search 5..8
2	5..8	6 → 42	Equal	Return 6

Precondition: the array must be sorted, or the comparison can't tell us which half is safe to discard.

Recursive Binary Search: Full Method

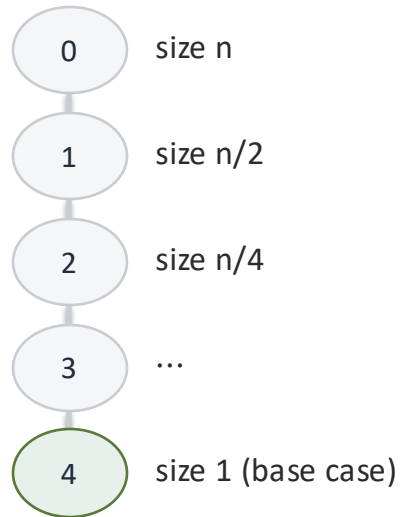
```
def binary_search(values, target):  
    return _binary_search(values, target, 0, len(values) - 1)  
  
def _binary_search(values, target, low, high):  
    if low > high:  
        return -1  
  
    middle = low + (high - low) // 2  
  
    if values[middle] == target:  
        return middle  
    if values[middle] > target:  
        return _binary_search(values, target, low, middle - 1)  
    return _binary_search(values, target, middle + 1, high)
```

The Recursion Tree



- height = $\log_2 n$ levels
- work per level = $O(1)$
- **total work = $O(1) \times \log_2 n = O(\log n)$**

The Recursion Tree



- height = $\log_2 n$ levels
- work per level = $O(1)$
- **total work = $O(1) \times \log_2 n = O(\log n)$**

Binary search makes ONE recursive call per level: the calls form a single chain, not a branching shape. (You can double-check this by counting how many times you can halve n before reaching 1: it's the same $\log_2 n$.)

Iterative Binary Search

```
def iterative_binary_search(values: list[int], target: int):  
    """Return the index of target in a sorted list, or -1 if absent."""  
    left = 0  
    right = len(values) - 1  
    while left <= right:  
        middle = (left + right) // 2  
  
        if values[middle] == target:  
            return middle  
        if values[middle] < target:  
            left = middle + 1  
        else:  
            right = middle - 1  
    return -1
```

Recursive vs. Iterative Binary Search

	Recursive	Iterative
Time	$O(\log n)$	$O(\log n)$
Extra space	$O(\log n)$ – the call stack	$O(1)$
Best for	Learning recursive reasoning	Production code

- Recursion shines when the data is itself recursive: trees, nested structures, divide-and-conquer.

Checkpoint 2: Trace It Yourself

2	5	9	14	20	27	35	44	58	71
0	1	2	3	4	5	6	7	8	9

target = 27

- What are the initial values of low, high, and middle?
- Which interval is searched in the second call?
- How many calls until the target is found?

Two to three minutes. Trace it on paper, then we'll check as a class.

PART 4

When Recursion Misbehaves: Naive Fibonacci

A cautionary tale: recursion that doesn't shrink fast enough

Naive Recursive Fibonacci

- fib: 0, 1, 1, 2, 3, 5, 8, 13, ...
- **Correct, but unlike binary search, this makes TWO recursive calls per level.**

```
def fibonacci_recursive(n):  
    if n <= 1:  
        return n  
  
    return fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2)
```

```
graph TD; 5((5)) --- 4((4)); 5 --- 3_3((3)); 4 --- 3_4((3)); 4 --- 2_4((2)); 3_4 --- 2_4a((2)); 3_4 --- 1_4a((1)); 2_4 --- 1_4b((1)); 3_3 --- 2_3((2)); 3_3 --- 1_3((1)); 2_3 --- 0_3((0)); 2_3 --- 1_3a((1)); 1_3 --- 0_3a((0));
```

23

Calls Grow Explosively: $O(2^n)$

n	Calls to fib(n)	Calls in binary search
5	15	3
10	177	4
20	21,891	5
30	2,692,537	5

Same $n = 30$: binary search needs about 5 calls; naive Fibonacci needs over 2.6 million. The number of recursive calls per level is what matters most.

Memoization: Remember, Don't Recompute

- Cache each result the first time it's computed.
- Next time the same subproblem appears, look it up instead of recomputing it.
- Turns fibonacci from $O(2^n)$ into $O(n)$.

This Has a Name: Dynamic Programming

- What memoization just did — solve each overlapping subproblem once, store the answer, reuse it — is a general strategy, not a fibonacci-only trick.
- It's called dynamic programming: break a problem into overlapping subproblems, solve each one exactly once, and build the final answer from what's already stored.
- *You'll study dynamic programming properly — knapsack, edit distance, shortest paths, and more — in a later algorithms course; for now, just recognize the shape.*

`fibonacci_recursive(30)`: **2,692,537 calls** → `fibonacci_memo(30)`: **59 calls** — same answer, 832,040.

Checkpoint 3: Why the Difference?

- Binary search and naive Fibonacci both recurse, but one is $O(\log n)$ and the other is $O(2^n)$.
- Turn to a neighbor: what is different about how many recursive calls each one makes per level?
- What would have to change about Fibonacci's recursive case for it to shrink the way binary search does?

Two minutes, pair-share, then we'll hear a few answers.

PART 5

Writing Your Own Recursive Functions

A practical checklist

Five Ways Recursion Goes Wrong

- **1. Missing or unreachable base case**
 - → calls continue until Python raises RecursionError.
- **2. The problem doesn't shrink**
 - → reusing middle instead of $\text{middle} \pm 1$ can repeat the same interval.
- **3. Forgetting to return the recursive result**
 - → the deeper call may find the answer, but it never travels back.
- **4. Binary search on unsorted data**
 - → discarding half is no longer logically valid.
- **5. Copying subarrays at every call**
 - → pass low/high indices instead — don't slice.

A Design Checklist for Recursion

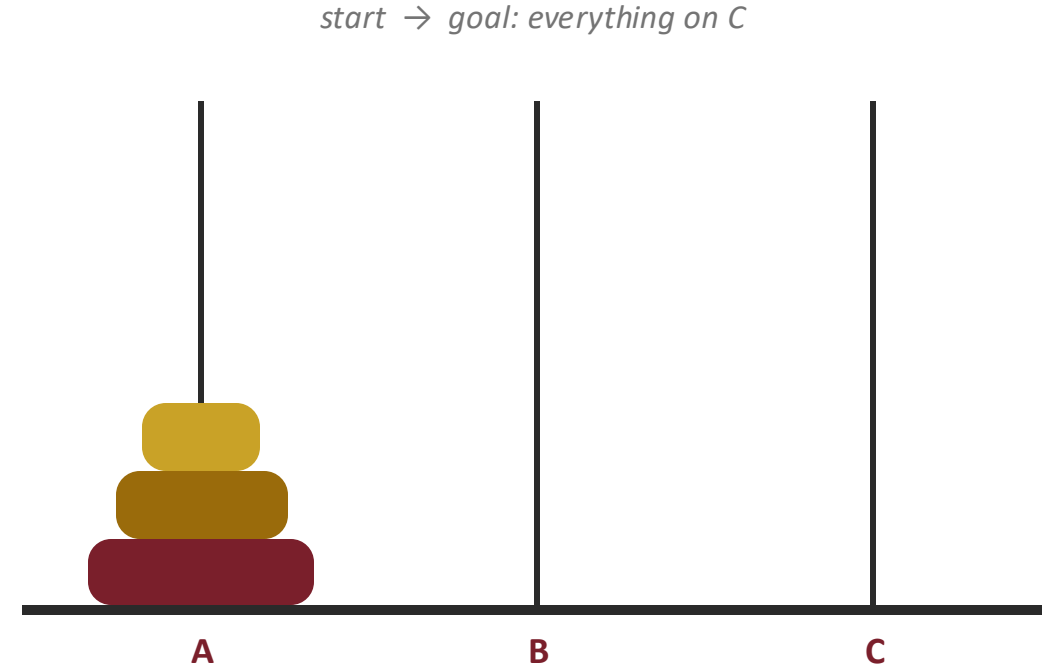
- **1. What does one call mean?**
 - State its contract precisely.
- **2. What is the smallest case?**
 - Can it be answered immediately?
- **3. How does a smaller instance of the same problem form?**
- **4. Why must repeated reductions reach the base case?**
- **5. What should the current call do with the returned result?**

Beyond These Two Examples

- Traversing linked structures and trees.
- Exploring graphs and mazes.
- Processing nested expressions or directories.
- Divide-and-conquer algorithms: merge sort, quicksort.
- Backtracking through a space of choices.
- **One call vs. many calls changes the entire complexity story — as we just saw.**

Exercise: The Towers of Hanoi

- Three pegs. n disks of different sizes, stacked smallest-on-top on peg A.
- Goal: move the whole stack onto peg C, using peg B as the spare.
- Rule: move exactly one disk at a time, and never set a larger disk on a smaller one.
- *Not a search, not a sort, not a tree — a different kind of problem for the same recursive thinking.*



The Recursive Solution

```
def hanoi(n, source, target, aux):  
    if n == 0:  
        return  
  
    hanoi(n - 1, source, aux, target)  
    move(source, target)  
    hanoi(n - 1, aux, target, source)
```

BASE CASE

RECURSIVE CASE

- Move the top $n - 1$ disks out of the way, move the one big disk, move the $n - 1$ disks back on top. That's the whole algorithm.
- $n = 3 \rightarrow 7$ moves. $n = 20 \rightarrow 1,048,575$ moves — the code above never changes.
- See `hanoi.py` for the verified move counts and a working `move()` that records each step.

Recursion doesn't just solve this — it makes the solution obvious. **An iterative version has to build its own explicit stack** to redo the bookkeeping the call stack already gives recursion for free.

Takeaways

- Recursion solves a problem through smaller instances of the same problem: base case, recursive case, progress.
- Each call gets its own stack frame; returns unwind in LIFO order.
- One recursive call per level chains (binary search, $O(\log n)$); more than one branches and can explode (Fibonacci, $O(2^n)$).
- **The number and size of recursive calls determines cost, not just correctness.**

Next class: recursion meets data structures. The Trainline / linked list, traversed both iteratively and recursively. Questions before we go?